Contamination Model: Technical Document

Oct 24, 2026 · @Ishan SIngh

1. Summary

This document explains how the MaverickX contamination decision-support system works: what data it reads, how it processes it, which algorithms it uses, how results are calculated and evaluated, and how they are shown. It describes the code in the `triple-ignition-challenge` repository as of the date above.

The system watches routine fermentation measurements (pH, temperature, dissolved oxygen, off-gas, biomass, product activity, and controller outputs where recorded). For every batch it answers four questions:

1. **Is this batch behaving unusually?** Each reading is compared with what approved normal batches looked like at the same hour.
2. **What is the most likely reason?** Contamination, slow but normal growth, a raw-material or process-condition change, a sensor fault, or an equipment fault, ranked by a transparent score.
3. **What should be checked next?** The cheapest, quickest check that best separates the leading explanations.
4. **Where might it have come from?** Medium lots, inoculum lots or bioreactors over-represented among contaminated batches.

What it does not do. It never declares contamination on its own (only a recorded independent test can confirm it), never gives a contamination probability, never identifies an organism from process signals, and never discards a batch. It supports a scientist's decision.

Headline results (details in section 11):

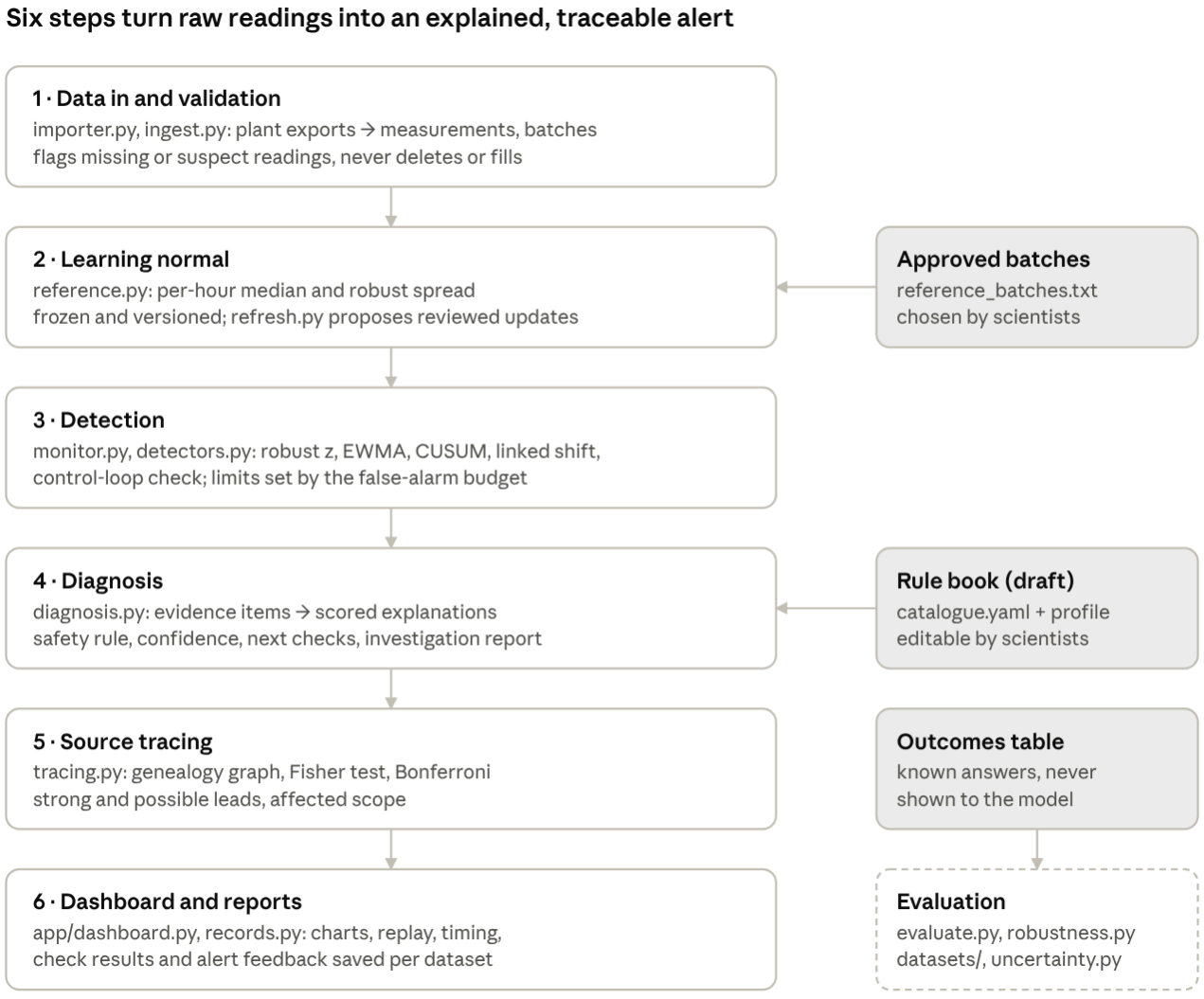
Test	Result
Own simulator, 500 held-out batches	120/120 contaminations caught, about 7.5 h before the simulated visual/odour check
IndPenSim (industrial-scale simulation), 10 equipment faults	10/10 caught, median 1.5 h after onset; 1/30 normal batches flagged; none called contamination
IndPenSim with a simulated contaminant, +20% respiration change	27/30 caught, about 19 h after onset

Test	Result
Real erythromycin plant, 163 later batches	26 flagged; flagged batches had lower final potency (Mann-Whitney p = 0.04)

None of the datasets contains a real contamination, so real-world contamination-detection performance is still unknown until MaverickX's batch records are available.

2. Architecture

The system is a pipeline of six steps; each is a separate module, and `pipeline.py` runs them end to end for a campaign folder.



architecture · six pipeline steps, their modules and inputs

Shaded boxes are inputs people control: the approved batch list, the rule book and the outcomes table. Evaluation (dashed) reads the pipeline's alerts and diagnoses together with the outcomes; the pipeline itself never reads the outcomes.

3. Frameworks and libraries

Everything is written in Python 3.13 (3.11 or newer works) as one package, `contamination`, plus a Streamlit app. Versions are pinned in `requirements.lock`; `requirements.txt` holds the runtime set used for Streamlit Cloud.

Tool	Version	Used for
pandas	3.0.6	All tables: measurements, batches, outcomes, scores, alerts, reports
NumPy	2.5.4	Vectorised statistics: robust z, EWMA, CUSUM, quantiles, linked-shift ranking
SciPy	1.18.1	Fisher's exact test (source tracing), Mann-Whitney test and normal quantiles (evaluation)
pandera	0.34.1	Structural schema checks on every input table (columns present, types parse)
NetworkX	3.7	Batch genealogy graph: cases linked through shared lots and equipment, clusters
scikit-learn	1.9.1	Comparison methods only (benchmarks), never in the live detectors
PyYAML	6.0.3	The rule book (`knowledge/catalogue.yaml`), process profiles and import mappings
Streamlit	1.66.0	The investigation dashboard (pages, forms, caching, file upload)
Plotly	7.1.0	Interactive dashboard charts: signal bands, timelines, genealogy graph, bar charts
Matplotlib	3.11.2	Static figures written by the demo for reports
openpyxl	3.1.5	Reading Excel exports in the importer
pytest	9.1.1	96 automated tests, including dashboard smoke tests with Streamlit's AppTest
ruff	0.17.1	Linting and formatting

No machine-learning model is trained. "Learning normal" means estimating robust statistics from approved batches (section 5), which keeps every number explainable. The PCA benchmark in `multivariate.py` is written directly in NumPy so its steps are visible.

4. Data model and processing

A *campaign* is a folder of plain CSV files. Measurements, batch genealogy and outcomes are kept in **separate tables** so the model can never see the answer it is judged on.

File	One row per	Key columns	Used by
`measurements.csv` (or `.csv.gz`)	reading	`batch_id`, `timestamp` (sample taken), `available_at` (result back), `variable`, `value`, `unit`, `source`, `quality_flag`, `observation` (operator note)	the model
`batches.csv`	batch	`batch_id`, `start_time` (inoculation), `bioreactor_id`, `medium_lot`, `inoculum_lot`	source tracing
`outcomes.csv`	batch	`category` (normal / contamination / non-contamination anomaly), `onset_h`, `reference_event_h` (when today's method noticed)	evaluation only, never the model
`investigations.csv`	check result	`batch_id`, `check_id`, `result` (positive / negative / inconclusive), `recorded_h`, `note`	diagnosis (independent evidence)
`reference_batches.txt`	approved batch	batch ids approved as normal	learning normal
`profile.yaml`, `variables.yaml`	campaign	which variable plays each role; process-specific variables	diagnosis rules, units

Ingestion and validation (`ingest.py`)

1. **Structure.** pandera checks that required columns exist and parse; a broken file stops with an error.
2. **Values are flagged, never deleted or filled.** Each problem sets `quality_flag` to `suspect` or `missing`, writes a reason in `quality_note`, and adds a row to an issues table: unrecognised flag, empty value, unknown variable or stage, unit that does not match the variable's unit (value not converted), value outside the plausible range, a result available before it was measured, and conflicting duplicates (all kept, all suspect). Exact duplicate rows are dropped and reported. Missing values stay missing; nothing is ever set to zero.
3. **Time.** `elapsed_h` counts from the batch's recorded start (inoculation), or from its first reading if no start is recorded. `available_h` is when the result existed. Every later step uses only readings with `available_h` at or before the hour being judged, so a lab result is never used before it came back.
4. **Only valid numbers are scored.** Flagged readings stay in the tables and appear as data-quality warnings on nearby alerts.

Plant data in any layout (`importer.py`)

Plant exports never arrive in this format. A `mapping.yaml` beside the exports says which file and column holds what: online signals in a wide layout (one column per signal), lab results in a long layout (one row per result, with sample and result times), operator notes, batch records and outcomes, from CSV or Excel. New variables are declared with a unit, a noise floor and a plausible range. `python -m contamination.importer convert` writes a campaign folder and lists every column the mapping did not use. `readiness` then reports which roles are measured, sampling interval and result delay per signal, approved normal batches (want 30 or more), confirmed contamination cases (want 10 or more), whether onset times and genealogy are recorded, and data-quality issues. `docs/data-template/` has worked examples.

5. Learning normal: the reference profile

The model's only "training" is a statistical description of approved normal batches, built once and frozen (Phase I of statistical process monitoring). It never sees the outcomes table.

For each variable *v* and each whole elapsed hour *h*, all valid readings from approved batches within ±1 h of *h* are pooled, and the reference stores:

```
m(v,h) = median of the pooled readings x
s(v,h) = max( 1.4826 × median|x − m(v,h)| ,  s_min(v) )
```

1.4826 × MAD equals the standard deviation for normally distributed data but is not pulled about by a few outliers. The floor s_min is the variable's measurement resolution (e.g. 0.02 for pH), so a perfectly flat reference never causes a division by zero. The number of batches covering each hour is also stored.

- **Coverage rule.** Where fewer than 5 reference batches cover an hour (for example a batch running far longer than normal), that hour is not scored and the batch summary reports "beyond the reference from *h*". On IndPenSim this removed two false contamination calls from unusually long batches.
- **Which batches.** The approved list comes from `reference_batches.txt` or, in the synthetic demo, the earliest 30 batches. IndPenSim uses 20 normal batches of each control strategy (60); the erythromycin plant the earliest 60% (243).
- **Frozen and versioned.** The profile stores its batch list, statistics, limits and calibration details, and a 12-character SHA-256 hash of its contents as its version. Every alert carries the reference version and algorithm version that produced it. Fitted profiles are cached (`reference_profile.json`) and reused only if the measurements, approved batches, settings and algorithm version are unchanged.
- **Refresh** (`refresh.py`). Processes drift, so a reviewed refresh proposes monitored batches that stayed within the reference with no problem outcome, shows how far each variable's normal trajectory and limits would move (in units of the current spread), and with a reviewer's note writes the new list and a line in `reference_history.csv` . Nothing is approved automatically.

6. Detection algorithms

Every valid reading is first turned into a robust z-score against the reference for its variable and hour. Five detectors then turn each batch's sequence of z-scores into a statistic, and an alert is raised wherever the statistic's size exceeds the detector's calibrated limit. None of these statistics is a probability of contamination.

```
z(t) = ( x(t) − m(v, h(t)) ) / s(v, h(t))
```

Detector	Statistic	Catches	Default limit before calibration
Robust threshold	z itself	One reading far from normal (spikes, sudden changes)	4

Detector	Statistic	Catches	Default limit before calibration
EWMA	exponentially weighted average of z, λ = 0.2	Small sustained drifts	$3\sqrt{(\lambda/(2-\lambda))} \approx 1.0$
CUSUM	two-sided cumulative sum of z beyond k = 0.5	Persistent shifts that build up	5
Linked shift	second-largest change from the batch's own past across a group of linked signals	Several linked signals changing together mid-batch, e.g. a contaminant raising CO_2 output and oxygen uptake and lowering dissolved oxygen	4
Control-loop check	z of a controlled variable while its controller does not push back	A pump, valve or cooling supply that stops responding	4

EWMA and CUSUM. With $E_0 = S^+_0 = S^-_0 = 0$, and missing readings leaving the state unchanged:

```
E(t)  = λ × z(t) + (1 − λ) × E(t−1)
S+(t) = max(0, S+(t−1) + z(t) − k)
S−(t) = max(0, S−(t−1) − z(t) − k)
```

The CUSUM statistic is S^+ when it dominates and $-S^-$ otherwise, so its sign gives the direction.

Linked shift. Batches differ from each other mostly by persistent offsets, and ordinary wobble usually moves one signal. For each signal in the process's linked group, the change from the batch's own recent past is

```
d(t) = z(t) − median{ z(u) : t − 36 h < u ⩽ t − 12 h }
```

(at least 3 earlier readings needed; the 12 h lag keeps a slowly growing change out of its own baseline). d is averaged over 6 readings, divided by its normal spread (1.4826 × MAD of the averaged change across approved batches), and the statistic is the **second-largest** absolute value across the group, so it only rises when two or more signals change together. Groups: CO_2 output, oxygen uptake and dissolved oxygen (IndPenSim and the erythromycin plant); dissolved oxygen and pH (synthetic α-amylase). The alert names the two signals that moved.

Control-loop check. For each recorded loop (pH held by the base pump, temperature held by cooling water), the controller output's z is read in the direction that would correct the current deviation (push). A healthy loop answers a drift by working harder:

```
L(t) = z_controlled(t)   if push(t) < 1
L(t) = 0                 otherwise
```

An optional single-signal shift detector (a CUSUM of d) exists but is off by default because it raised flagged batches on the real plant from 9% to 39%.

From statistics to alerts

- **Alert episodes.** Consecutive readings over the limit, for one batch, variable and detector, form one alert with its start and end hours, when it could first have been raised (the result-availability time), the number of points, *isolated* (one reading) or *persistent*, direction, peak statistic and limit, observed value versus reference median, robust z at the peak, and the reference and algorithm versions.
- **Context.** Each alert lists corroborating variables (others alerting within ±3 h) and data-quality warnings (flagged readings within ±3 h, or a reference built from few batches at that hour).
- **Batch summary.** *Substantially abnormal* when two or more variables have persistent alerts, *abnormal* with any alert, otherwise *within reference*. Process deviation is reported separately from contamination evidence.
- **Replay.** Monitoring can run "as of" any hour using only results available by then: the dashboard's replay and the timing evaluation use this.

7. Calibration: how tight the limits are

Each detector's limit is set per variable from the approved batches themselves, so that normal batches would not have alerted, and is then frozen into the reference profile.

1. **Leave-one-out peaks.** Each approved batch is scored against a reference built *without* it, which gives realistic out-of-sample deviations. For every detector and variable, its peak |statistic| over the batch is recorded.
2. **Limit.** limit = max(default limit, 1.1 × a quantile of those peaks). The starting quantile is 1.0, i.e. 10% above the single most extreme approved batch.
3. **False-alarm budget.** One unusual approved batch would set the limit for everyone, so the limits of the robust threshold and the linked-shift detector are tightened in steps of 0.5 percentile points, each until the approved batches it alone would have alerted on reach its share of the budget. The default budget is **5% of approved batches, split**

equally between the two. EWMA, CUSUM and the control-loop check stay at the most extreme batch: tightening EWMA and CUSUM mostly added false alarms on IndPenSim, and the loop check separates faults without tightening.

The budget is a batch-level rate, so it does not grow with the number of variables, and it is the setting the plant chooses: how many normal batches in 100 they can afford to review. On a stable process the held-out false-alarm rate tracked it (IndPenSim: 1/30 at 5%, 3/30 at 10%); on a drifting one it can be exceeded (the erythromycin plant). The dashboard's slider offers 1, 2, 5, 10 and 20%. The profile records the quantiles chosen and which approved batches would have alerted, so the cost of each setting is visible.

A fixed percentile was tested and rejected: the same "95th percentile" that caught every IndPenSim fault flagged 60% of the erythromycin plant's batches.

8. Diagnosis: from alerts to explanations

For every batch with alerts (or recorded checks), the system computes evidence items, scores six explanations with a rule book, and recommends checks. The rule book is a YAML file (`knowledge/catalogue.yaml`) that scientists can read and edit without touching code; it is a **draft** that MaverickX scientists have not reviewed. A process profile maps its roles (biomass, oxygen, pH, product, temperature, pH controller, temperature controller) to each process's variables, so the same rules run on α-amylase, penicillin and erythromycin data.

Evidence items

Each item is **present**, **absent** or **unknown** (not enough data yet). Unknown never counts for or against anything.

Item	Present when
E01 Several signals deviate together	3+ variables (not temperature) have persistent alerts, or a persistent linked-shift alert
E02 Only one signal deviates	all alerts are on one variable
E03 Isolated readings that return to normal	threshold alerts are single readings and the variable is back within reference
E04 Delayed but otherwise normal trajectory	the growth-phase biomass curve fits the reference shifted 3 h or more later, much better than unshifted
E05 Biomass above reference	persistent high alert on biomass

Item	Present when
E06 Low product for the biomass	product clearly below reference while biomass is normal or high (time-shifted if the batch is delayed)
E07 Culture more acidic	persistent low alert on pH
E08 Extra oxygen demand with high biomass	dissolved oxygen clearly low while biomass is clearly high, at 2+ readings
E09 Deviation decoupled from linked signals	pH, oxygen or biomass deviates for 6 h+ while the other two stay normal 80%+ of the time
E10 Visual / odour observation	an operator note is recorded
E11 Data-quality problems	flagged readings near the alerts
E12 / E13 Confirmatory test positive / negative	a recorded microscopy, plating or qPCR result
E14 Process condition deviation	persistent temperature alert
E15 / E16 Instrument check fault / no fault	a recorded probe check
E17 Controller working hard	a controller output deviates persistently while its variable stays in range
E18 Controller not responding	a controlled variable drifts persistently while, at the start of the drift, its controller output does not push back
E19 / E20 Equipment check fault / no fault	a recorded actuator check

Explanations and scoring

```
score(H) = sum of weights of H's supporting items that are present
         − sum of weights of H's contradicting items that are present
```

Weights are 1 (weak), 2 (moderate) or 3 (strong). The score orders explanations for review; it is not a probability.

Explanation	Supported by (weight)	Can only lead with	Contradicted by (weight)
H01 Biological contamination	E05 2, E08 2, E12 3, E01 1, E06 1, E07 1, E10 1, E17 1	E05, E08, E10 or E12	E13 3, E03 2, E04 2, E09 2, E18 2

Explanation	Supported by (weight)	Can only lead with	Contradicted by (weight)
H02 Expected variation (slow growth)	E04 3	E04	E12 3, E05 2, E06 1, E08 1, E10 1
H03 Raw-material or condition change	E14 2, E01 1, E17 1	E14	E12 2, E18 2, E03 1, E09 1
H04 Measurement fault (sensor/probe)	E03 3, E09 3, E15 3, E02 1, E11 1	E03, E09 or E15	E01 3, E16 3, E12 2, E18 2, E10 1
H06 Equipment or actuator fault	E18 3, E19 3	E18 or E19	E20 3, E12 2
H05 Unresolved / previously unseen	(fallback)		

An explanation is **leading** only if it scores highest, at least 2, at least 1 point clear of the next, and has one of its specific items present; otherwise the leading answer is H05, "unresolved, too early to tell", which is the safe failure. Others are *possible* (positive score) or *not supported*. The specificity rule stops non-specific evidence ("several signals deviate") from ever naming contamination on its own.

Assessment, kept as separate dimensions

- **Contamination evidence:** "confirmed by independent test", "independent test negative", or "unconfirmed". Only E12 can confirm.
- **Confidence:** *strong* when the leading explanation rests on an independent test; *moderate* when it leads by 2+ points with 2+ supporting items and nothing contradicting; otherwise *limited*.
- **Severity:** process deviation (section 6), quality impact (product below reference or not), and affected scope (from source tracing).

Next checks

The two highest-scoring explanations (at least two are always compared), plus contamination while it has not been ruled out by a negative test (the **safety rule**: missing a contamination is the costly error), form the focus. Each check in the catalogue states which explanations it tells apart, its turnaround and its relative cost. Checks are ranked by how many focus explanations they inform, then cost, then turnaround; the top three are shown, skipping checks already done and screens once contamination is confirmed.

Check	Informs	Turnaround
T01 Microscopy / Gram stain	contamination, slow growth, condition change	~1 h
T02 Probe vs offline measurement	measurement fault	~0.5 h
T03 Batch record review	slow growth, condition change	~0.5 h
T04 ATP bioluminescence (screen)	contamination	~0.25 h
T05 Plate count on selective medium	contamination	~36 h
T06 Targeted qPCR	contamination	~4 h
T07 Actuator check (pumps, valves, cooling)	equipment fault, condition change	~0.5 h

Turnarounds and costs are placeholders until MaverickX supplies its own. A recorded result becomes evidence (e.g. T01 positive → E12) and the assessment updates immediately. The investigation report (Markdown, downloadable) has seven parts: what changed, ranked explanations with supporting and contradicting evidence, missing information, the assessment, possible sources, next checks, and a scientist-decision block.

9. Source tracing

When several batches are contaminated, source tracing asks what they had in common that clean batches did not. It reports **leads for investigation, never proof of cause** (`tracing.py`).

1. **Status per batch** from its diagnosis: a *case* (contamination confirmed by test, or suspected as the leading explanation), a *non-case* (approved reference batch, no alerts, another explanation leading, or contamination ruled out by test), or *uncertain* (contamination only possible, or unresolved). Uncertain batches count neither for nor against a resource.
2. **Genealogy graph** (NetworkX): a bipartite graph linking each batch to the medium lot, inoculum lot and bioreactor it used. Cases connected through shared resources form clusters; a case sharing nothing with other cases is reported as possibly independent.
3. **Statistics per resource** used by at least 2 cases: the attack rate (cases among its classified users), the attack rate among non-users, the relative risk (their ratio), and a one-sided Fisher's exact test of "cases are over-represented among users", Bonferroni-corrected over all such resources (family-wise $\alpha = 0.05$).

4. **Tiers.** *Strong lead*: significant after correction with an attack rate of 50% or more. *Possible lead*: shared by 2+ cases with a relative risk of 2 or more. *Not over-represented*: shared, but cases are no more common among its users. *Single case*.
5. **Confounding check.** A further strong lead must stay significant once batches that used a stronger lead are excluded; otherwise it is downgraded with a note. This stops an inoculum lot that merely overlaps the contaminated medium lot from looking like a second source.

Each lead carries a suggested check (e.g. test a retained sample of the medium lot; streak-plate the inoculum stock; swab the bioreactor and review cleaning records). The report's affected scope lists linked cases and any other batches that used a strong-lead resource. Tracing only sees the recorded genealogy; water, transfer lines, operators and environmental samples are not modelled.

10. Evaluation method

Results are only meaningful if the model never sees the answers and is tested on batches it was not tuned on. Four datasets test different things:

Dataset	What it is	Reference (approved normal)	Tested on	What it tests
Synthetic α-amylase (`synthetic.py`)	Our 48 h simulator: logistic growth, oxygen demand tied to growth, product in stationary phase; pH, temperature and DO hourly, OD every 2 h (0.5 h delay), activity every 4 h (3 h delay). Scenarios: normal, slow growth, contamination (onset 14–24 h, visual/odour note 12–18 h later), pH sensor spike, DO probe drift; ~20% of batches have missing-data gaps; medium lot M-12 planted in 4 of 6 contaminated batches	earliest 30 batches	25 later batches × 20 fresh seeds (11–30) = 500	the whole system, including contamination and source tracing
IndPenSim (Goldrick et al. 2015, CC BY 4.0)	100 simulated 100,000 L penicillin batches, 167–290 h, three control strategies, labelled equipment faults in batches 91–100	20 normal batches per strategy (60)	30 other normal + 10 fault batches	false alarms on realistic variation, fault detection and diagnosis (faults are not contamination)

Dataset	What it is	Reference (approved normal)	Tested on	What it tests
IndPenSim + simulated contaminant (`datasets/contaminate.py`)	copies of the 30 held-out normal batches with a contaminant's effects added from a random onset, at 4 strengths	as above, untouched	30 normal + 120 contaminated	how large a contaminant must be to be caught on realistic variation
Erythromycin plant (Zenodo 14619074, CC BY 4.0)	406 real industrial batches from 2022, hourly; no labels; final potency held out as the outcome	earliest 60% (243)	later 163	whether flags point at worse batches on real data

Rules. Splits are by whole batch (never by reading) and chronological where time matters. Outcomes are read only by the evaluation code. Settings were tuned on synthetic seeds 1–10 and results reported on seeds 11–30. Changes developed while looking at IndPenSim (the coverage rule, the control-loop check, the linked-signal detector) are disclosed as such: those numbers are no longer unseen.

Metrics.

- **Flagged after onset**: a problem batch counts as caught only if an alert starts at or after the onset; earlier alerts are counted as false alerts.
- **Sensitivity / specificity**: problem batches caught; normal batches with no alert.
- **Onset to first alert** (speed) and **lead time** = when today's method noticed (the simulated visual/odour note) minus the first alert (time saved).
- **False alerts per batch**: alert episodes on normal batches, or before onset.
- **Diagnosis**: the leading explanation compared with the known scenario at the first post-onset alert, +4 h, +8 h and end of batch, as *matched*, *unresolved* (the safe failure) or *wrong* (pointing the investigation the wrong way).
- **Source tracing**: whether the planted lot is the top strong lead, how many planted cases it links, and false strong leads.
- **Erythromycin**: flag rate among low-potency batches (below the reference's 10th percentile) versus the rest (one-sided Fisher's exact test), and final potency of flagged versus unflagged batches (Mann-Whitney U).

Uncertainty. Samples are small, so every count is reported with a 95% Wilson score interval (e.g. 8/10 is 49–94%) and medians with a 5,000-resample bootstrap interval over batches (`uncertainty.py`).

Benchmark. A standard multivariate method (PCA with Hotelling's T^2 and SPE, 90% of variance, 5-fold cross-validated limits under the same budget) is run on the same hourly signals for comparison only (`multivariate.py`).

11. Results

On realistic data the system catches equipment faults within hours without mistaking them for contamination, and catches a contaminant once it changes respiration by about 20%. The perfect synthetic scores reflect a simulator that shares the rules' assumptions. Full tables: `docs/mvp-results.md` and `docs/external-validation.md` .

Synthetic α-amylase, seeds 11–30 (500 test batches)

Measure	Result
Contaminated batches flagged after onset	120/120 (97–100%)
Sensor spikes and probe drifts flagged	120/120
Normal-category batches flagged (includes 60 slow-growth batches)	81/260 (26–37%)
Median onset to first alert	~7.8 h
Median lead over the simulated visual/odour check	7.5 h (per-seed medians 5–9 h)
Right explanation 8 h after the first alert	241/300 (6 wrong, the rest unresolved)
Right explanation at end of batch	295/300 (3 unresolved, 2 wrong); contamination 120/120
Planted medium lot found as top strong lead	20/20 seeds; 1 false strong lead in 20 (a genuine chance cluster)

IndPenSim equipment faults (40 held-out batches)

Measure	Result
Fault batches caught	10/10 (72–100%), median 1.5 h after onset

Measure	Result
Normal batches flagged	1/30 (1–17%)
Wrongly called contamination	0/40 (0–9%)
Read as equipment or actuator fault	4/10: all pump and cooling faults
By fault	aeration 0 h; substrate feed 2 h; base pump 1 h; cooling 9–10 h

IndPenSim with a simulated contaminant

Change in respiration	Caught	Onset to alert	Named as contamination
+10%	1/30	–	0/30
+20%	27/30 (74–97%)	19 h	4/30
+40%	30/30	13 h	29/30 (at ~26 h)
+80%	30/30	11 h	30/30 (at ~22 h)

The 30 untouched copies: 1/30 flagged, 0/30 called contamination. The first sign was almost always CO_2 output.

Erythromycin plant (163 later batches, real data)

26 batches flagged (16%), none called contamination. Flagged batches had lower final potency (median 10,690 vs 10,850; Mann-Whitney p = 0.04); low-potency batches were flagged 10/48 versus 16/115 for the rest (Fisher p = 0.19). Without the linked-signal detector it flagged 14 (p = 0.001): the extra flags are genuine respiration changes that did not hurt potency. 2,016 logger artefacts were flagged at ingestion, none deleted.

Benchmark

PCA with T^2/SPE under the same budget caught 7/10 IndPenSim faults (ours 10/10) and matched our detectors on the synthetic data (120/120 at ~7.7 h), so the simpler, explainable detectors were kept.

12. How results are displayed

Scientists work in a Streamlit dashboard (`app/dashboard.py`); every number on it comes from the same pipeline functions the evaluations use. The sidebar picks the dataset and the false-alarm budget, shows the reference version and how it was approved, and holds

the saved-records panel. A banner says what kind of data is shown (synthetic, IndPenSim or the real plant), and a "Show known outcomes" toggle reveals the evaluation answers, which the model never sees.

Page	What it shows and how it is calculated
Campaign overview	One row per monitored batch: status (from source tracing), process deviation, first alert hour, variables flagged, leading explanation, contamination evidence, confidence, affected scope and the top next check; headline counts above
Batch investigation	One chart per measured variable: the observed trajectory over the reference band (median ± 3 robust SD) and median, alert readings marked, gaps in the data left as gaps. Cards for the leading explanation, contamination evidence, process deviation and confidence; tabs for explanations with their scores and evidence, the alerts table (detector, direction, times, observed vs reference, robust z, corroborating variables, linked-signal detail, data-quality warnings), evidence items, next checks and possible sources, the full report
Replay	A slider re-runs monitoring and diagnosis "as of" any hour, using only results available then
How it developed	The batch replayed at regular hours from the first alert to the end: a step chart of the leading explanation over time, with the first alert (and, with known outcomes, the true onset and today's method) marked
Record a check / alert feedback	Forms that add a check result (which becomes evidence immediately) or a verdict on an alert (real issue, false alarm, not sure)
Detection timing	For batches with a known onset: hours from onset to first alert and lead over today's method, as bars, with the false-alarm rate beside them

Page	What it shows and how it is calculated
Source tracing	The genealogy graph of cases and the resources linking them (strong leads orange, possible leads grey), clusters, and the candidate table: cases, attack rates, relative risk, p-values, suggested check
Value estimate	Yearly value from the plant's own inputs (`value.py`): later batches spared by finding a shared source, doomed batches stopped earlier, expensive tests avoided, good batches not scrapped, minus the cost of checking false alarms; the lead time and false-alarm share are pre-filled from the campaign, all costs are placeholders
Load your data	Upload exports and a mapping, convert, view the readiness report, then monitor the result
How it works	A plain-language explanation and the limits

Saving. Check results and alert feedback are saved per dataset in `data/records/` (outside git) and reloaded in new sessions; the panel downloads them and restores them from a file, which is needed on Streamlit Cloud because its disk is wiped on restart.

Speed. Fitted references are cached on disk and in memory; a new budget recalibrates once (seconds on the synthetic demo, a few minutes on the erythromycin plant).

Outside the dashboard. `python -m contamination.demo` writes alerts, batch summaries, detector comparisons, diagnoses, source candidates, the reference profile, one Markdown investigation report per flagged batch (`docs/example-investigation-report.md` is one) and figures to `outputs/`.

13. Design rules and safeguards

These rules, agreed at the start of the project, shape every part of the code:

1. **Scores are not probabilities.** No anomaly score or explanation score is ever presented as a probability of contamination, and no organism is identified from pH, temperature or oxygen alone.
2. **Evidence and deviation stay separate.** Process deviation and contamination evidence are separate fields, and so are confidence and severity.
3. **No label leakage.** Outcomes live in their own table, read only by evaluation code; splits are by whole batch; only data available at the hour being judged is used.

4. **Missing stays missing.** Values are never filled with zero; unusual values are flagged, never silently dropped.
5. **Simple and explainable first.** Robust thresholds, then EWMA and CUSUM, then targeted extensions; multivariate and machine-learning methods are comparisons only.
6. **Structured alerts.** Every alert records the variable, observed versus reference, magnitude, detector, reference and algorithm versions, data-quality warnings, persistence and confirmation status.
7. **Only a test confirms.** Contamination is "confirmed" only by a recorded independent test; the safety rule keeps a contamination check on the list until one rules it out.
8. **Honest labels.** Results on synthetic data are labelled software demonstrations, and changes developed while looking at a test set are disclosed.
9. **Reviewable knowledge.** Rules, weights, checks and process mappings live in YAML for scientists to review, marked as a draft until they have.

14. Limitations and next steps

The system has not yet seen a real contamination; its contamination performance on MaverickX's process is unknown until their batch records arrive.

- **Assumed contaminant effects.** Both contamination tests use effects we wrote; real contaminants may differ by organism, timing and process.
- **A detection threshold.** On realistic variation a contaminant that changes respiration by about 10% is not seen; at about 20% the alert usually comes with the cause "unresolved" until the 12-hourly biomass result arrives.
- **Review load on real plants.** The erythromycin plant flagged 16% of later batches; the budget held on a stable process but was exceeded on a drifting one.
- **Draft rule book.** Weights, checks, turnarounds and costs are ours, pending scientist review.
- **Limited genealogy.** Tracing sees medium lot, inoculum lot and bioreactor only; small campaigns produce chance clusters.
- **Investigation, not prediction.** It does not forecast contamination before it starts.

Next steps: import MaverickX's data with the importer and run the readiness report; set the false-alarm budget with them; run a blind test on held-back batches including confirmed contaminations; have their scientists review the rule book and supply real check times and costs; prioritise off-gas CO_2/oxygen data and more frequent biomass readings, which most lower the detection threshold; refresh the reference periodically; keep real plant data off the public demo deployment.

15. Appendix

Running it

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.lock && pip install -e ".[dev]"
python -m contamination.demo                                    # synthetic
campaign, reports, figures
streamlit run app/dashboard.py                                  # dashboard at
http://localhost:8501
pytest                                                          # 96 tests
python -m contamination.robustness --seeds 11 31                # synthetic
evaluation, 20 seeds
python -m contamination.datasets.indpensim evaluate             # IndPenSim
faults
python -m contamination.datasets.contaminate                    # IndPenSim +
simulated contaminant
python -m contamination.datasets.erythromycin evaluate          # real plant
python -m contamination.multivariate indpensim                  # PCA
benchmark
python -m contamination.importer convert mapping.yaml --out
data/maverickx/campaign
python -m contamination.importer readiness data/maverickx/campaign
python -m contamination.refresh data/erythromycin               # propose a
reference refresh
```

Code map (`src/contamination/`)

Module	What it does
`schema.py`	Variables (unit, noise floor, plausible range) and table schemas
`ingest.py`	Loading and validation; flags, never deletes
`importer.py`	Plant exports → campaign via `mapping.yaml`; readiness report
`reference.py`	Robust per-hour reference; versioning
`detectors.py`	Robust z, EWMA, CUSUM, own-past change, control-loop statistic
`monitor.py`	Scoring, linked-shift and loop statistics, calibration and budget, alerts, batch summary

Module	What it does
`alerts.py`	The structured `Alert`
`diagnosis.py`	Evidence, scoring, assessment, next checks, reports
`knowledge/catalogue.yaml`, `knowledge/profiles/`	Rule book and process profiles
`tracing.py`	Genealogy graph, Fisher/Bonferroni leads, confounding check
`pipeline.py`	Campaign loading, cached fitting, end-to-end run, replay, timeline
`refresh.py`	Reference refresh and history
`records.py`	Saved check results and alert feedback
`value.py`	Value estimate
`synthetic.py`, `demo.py`	Simulator and demo run
`evaluate.py`, `robustness.py`, `uncertainty.py`, `multivariate.py`	Evaluation, multi-seed runs, intervals, PCA benchmark
`datasets/`	IndPenSim, erythromycin and semi-synthetic contamination

Glossary

Term	Meaning
Approved / reference batch	A batch scientists accept as normal, used to learn "normal"
Robust z	How many robust standard deviations a reading is from the reference median at that hour
MAD	Median absolute deviation; 1.4826 × MAD estimates the standard deviation without being pulled by outliers
EWMA / CUSUM	Smoothed average / cumulative sum of deviations, for drifts and persistent shifts
Leave-one-out	Scoring each approved batch against a reference built without it

Term	Meaning
False-alarm budget	The share of approved normal batches allowed to alert; sets how tight the limits are
Onset	When a problem actually started (known only in simulations or from records)
Lead time	How much earlier than today's method the first alert came
Attack rate / relative risk	Share of a resource's users that are cases / that share divided by the share among non-users
Bonferroni correction	Dividing the significance level by the number of resources tested, so chance leads are rarer
Wilson interval	A 95% range for a proportion that behaves well at 0/n and n/n
IndPenSim	Published simulation of an industrial penicillin process (Goldrick et al. 2015)
OUR / CER	Oxygen uptake rate / carbon dioxide evolution rate (respiration)